



密码分析学实验报告

姓名: 邱泽辉、江李阳、王健一

学号: 202400460046、202400460042、202400460037

专业: 密码科学与技术

日期: October 3, 2026

Contents

1	实验目的	2
1.1	组员分工	2
2	实验环境	2
3	实验原理	2
3.1	差分路线	2
3.2	差分	2
3.3	CipherFour 算法	3
3.4	利用线性规划寻找差分路线	3
4	任务一	4
4.1	直接穷举思路	4
4.2	S 盒差分分布表	5
4.3	一轮差分传播	5
4.4	四轮动态规划	6
4.5	高概率差分的搜索	7
4.6	方法 2	8
4.7	第一轮不同	9
4.8	找到其他路线	9
5	任务二	11
6	任务三	11
6.1	实验目标与方法	11
6.2	程序实现	12
6.3	实验结果	13
6.4	结果分析	14
7	任务四	14
7.1	实验目标与攻击思路	14
7.2	明文配对与去噪条件	15
7.3	六组密钥下的去噪实验	15
7.4	最后一轮部分解密与密钥评分	16
7.5	复杂度分析	16
8	实验总结	17
A	实验代码	18
A.1	寻找差分路线	18
B	实验报告完整地址	22

1 实验目的

1.1 组员分工

2 实验环境

本实验在个人计算机上完成，使用 Python 编写 Enigma 密码机的模拟和性能测试程序。具体环境如下：

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约为 2.1 GHz；内存 16 GB。
- 软件环境：Windows 操作系统、MacOS, Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。

3 实验原理

3.1 差分路线

i 轮差分路线是指对输入对 (X, X') ，在 i 轮加密过程中满足对应第 j 轮的一对输出值 (Y, Y') ($1 \leq j \leq i$)，满足 $\delta X = X \oplus X' = \beta_0$ ， $\delta Y = \oplus Y' = \beta_j$ ($1 \leq j \leq i$)，则称

$$\beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_i$$

为一条 i 轮差分路线。假设密码算法的输入 X 和各轮密钥 K_i 的取值是相互独立且均匀分布的，对于满足输入差分为 β_0 、输出差分为 β_i (显然满足输入差分为 β_0 的明文不只一对) 的 i 轮差分路线概率为 $DP(\beta_0 \rightarrow \beta_i)$ ，其概率为各轮差分路线概率的乘积

$$DP(\beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_i) = \prod_{(j=1)}^i DP(\beta_{j-1} \rightarrow \beta_j)$$

注意这里差分路线是精确记录了包括起始、最终结果和中间每一种状态的一条路线。影响上述 i 轮差分路线的概率的主要因素是输入差分非零的 S 盒的个数以及输入差分非零的 S 盒对应的输出差分 (因为当前轮 S 盒的输出即为下一轮 S 盒的输入)，其中输入非零的 S 盒被称为活跃 S 盒。

3.2 差分

i 轮差分是指给定了输入差分 β_0 和 i 轮加密后的输出差分 β_i ，满足上述条件的称为 i 轮差分。与 i 轮差分路线相比，它不指定加密的中间状态。因此，对于 i 轮差分的概率计算如下：假设满足输入差分为 β_0 且 i 轮之后输出差分为 β_i 的 i 轮差分路线共 s 条，则：

$$\begin{aligned} DP(\beta_0 \xrightarrow{i \text{ 轮}} \beta_i) &= \sum_{t=1}^s DP(\beta_0 \xrightarrow{1 \text{ 轮}} \beta_1^t \rightarrow \cdots \beta_{i-1}^t \xrightarrow{1 \text{ 轮}} \beta_i) \\ &= \sum_{t=1}^s \left(\prod_{j=1}^i DP(\beta_{j-1}^t \xrightarrow{1 \text{ 轮}} \beta_j^t) \right) \end{aligned}$$

也就是说 i 轮差分的概率是由所有满足输入和输出条件的 i 轮差分路线共同组成的，它的概率是要大于等于单条差分路线的概率的，而且想要精确计算差分的概率，还是要回到计算每一条差分路线的概率上去。

3.3 CipherFour 算法

CipherFour 五轮加密流程

16 比特分组 · 四个并行 4×4 S 盒 · 六个轮密钥

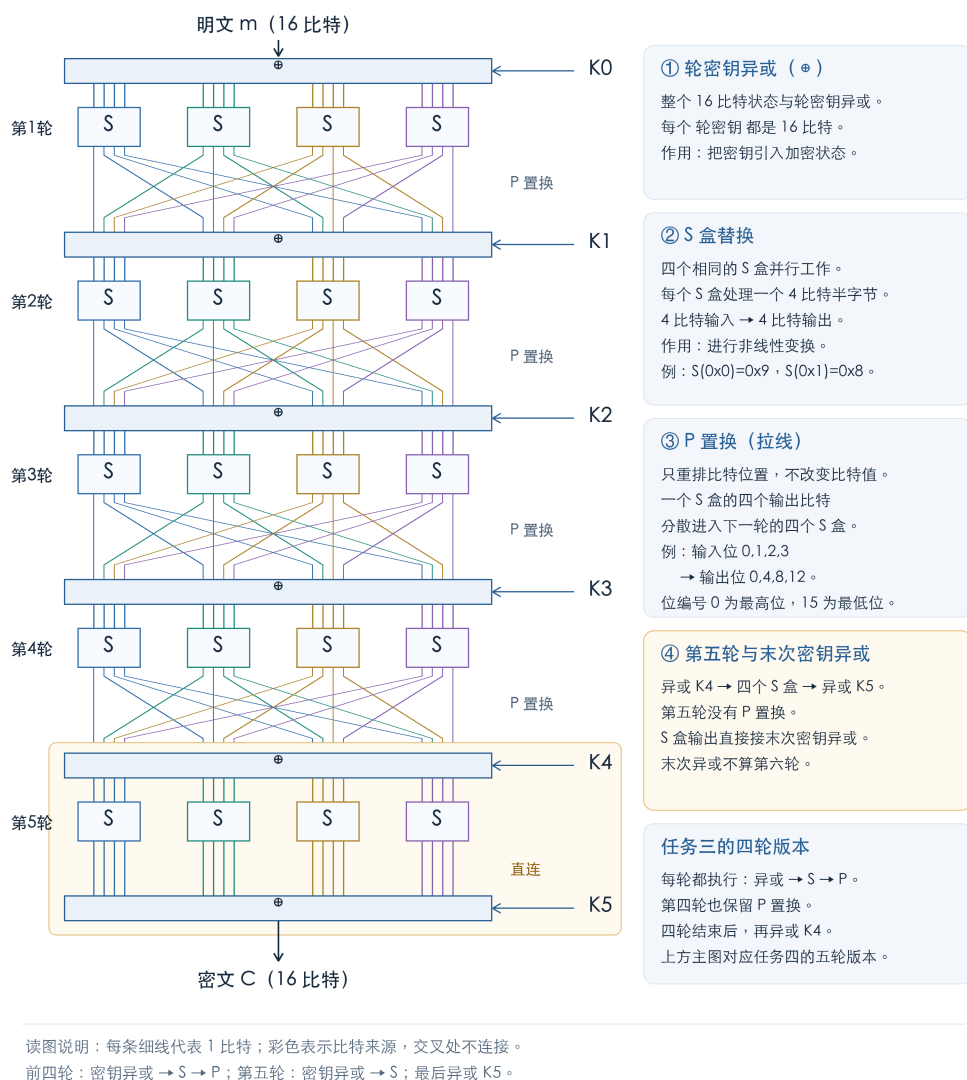


Figure 1: CipherFour 算法 5 轮加密流程图

3.4 利用线性规划寻找差分路线

定义 $B_r(\Delta)$ 为经过 r 轮后，到达差分状态 Δ 的所有路线中，概率最大的一条的概率。对于输入差分，把输入差分为 0 直接丢掉，因为输入差分为 0 之后的概率都是 1，只能

找到全 0 的路线，定义

$$B_0(\Delta) = \begin{cases} 1, & \Delta \neq 0, \\ 0, & \Delta = 0. \end{cases}$$

沿用书上的符号，假设每一轮的差分为 β_r

假设已经知道了 $B_{r-1}(\beta_{r-1})$ ，现在要计算 $B_r(\beta_r)$

从前驱 β_{r-1} 到达 β_r 的最佳完整路线概率为

$$\Pr(\beta_{r-1} \rightarrow \beta_r) = B_{r-1}(\beta_{r-1}) \cdot \Pr(\beta_{r-1}, \beta_r).$$

然后再遍历所有前驱，就有

$$B_r(\beta_r) = \max_{\beta_{r-1}} \{B_{r-1}(\beta_{r-1}) \cdot \Pr(\beta_{r-1}, \beta_r)\}$$

这在代码上看是比较简单的，只要遍历所有 β_{r-1} 就行， $B_{r-1}(\beta_{r-1})$, $\Pr(\beta_{r-1}, \beta_r)$ 知道 β_{r-1} 后可以直接查表

但是理论上的东西，并不总是可行的，要知道每个 β_{r-1} 可是有 2^{16} 个，看上去也不是很多，但是注意这里的时间复杂度是在指数上的，也就是 $O(2^{16r})$

那么如果是 4 轮，就会达到 2^{64} ，那这就不是一个小数目了，所以要拆分成 4 个 S 盒

4 任务一

««««< HEAD

本任务要求寻找除题目给定差分之外的高概率四轮差分。题目给出的差分为

$$(0x0, 0x0, 0x0, 0x1) \xrightarrow{4 \text{ 轮}} (0x0, 0x0, 0x0, 0x1),$$

也可以写成

$$0x0001 \xrightarrow{4 \text{ 轮}} 0x0001.$$

4.1 直接穷举思路

对于固定的输入差分 α ，可以遍历所有 2^{16} 个 16 比特明文 x ，构造有序明文对 $(x, x \oplus \alpha)$ ，然后分别进行四轮 CipherFour 加密。若最终密文差分为 β ，则将对对应计数加一。因此，固定输入、输出差分的概率为

$$P(\alpha \rightarrow \beta) = \frac{\#\{x : E(x) \oplus E(x \oplus \alpha) = \beta\}}{2^{16}}.$$

对于每一个非零输入差分，共有 2^{16} 个有序明文对；如果不区分明文对的顺序，则有 2^{15} 个无序明文对。若对全部 $2^{16} - 1$ 个非零输入差分都进行穷举，需要处理约 2^{32} 组明文对，计算开销较大。因此，程序使用 S 盒差分分布表（DDT）对差分进行动态规划。

4.2 S 盒差分分布表

CipherFour 使用一个 4 进 4 出的 S 盒。对于输入差分 α 和输出差分 β ，DDT 定义为

$$\text{DDT}[\alpha][\beta] = \#\{x \in \{0, 1, \dots, 15\} : S(x) \oplus S(x \oplus \alpha) = \beta\}.$$

因此，单个 S 盒的差分传播概率为

$$T[\alpha][\beta] = \frac{\text{DDT}[\alpha][\beta]}{16} = \frac{\text{DDT}[\alpha][\beta]}{2^4}.$$

代码中的 DDT 计算如下：

```
1 for diff_in in range(16):
2     for value in range(16):
3         diff_out = S[value] ^ S[value ^ diff_in]
4         diff_table[diff_in][diff_out] += 1
```

其中，`diff_table[diff_in][diff_out]` 保存输入差分为 `diff_in`、输出差分为 `diff_out` 的出现次数。

4.3 一轮差分传播

一个 16 比特差分可以拆成四个半字节：

$$\Delta = (a_0, a_1, a_2, a_3).$$

若四个 S 盒的输出差分为

$$\Gamma = (b_0, b_1, b_2, b_3),$$

则在固定当前差分的条件下，四个 S 盒同时发生该传播的概率为

$$Q(\Delta, \Gamma) = \prod_{i=0}^3 T[a_i][b_i].$$

例如，输入差分 $0x0001 = (0, 0, 0, 1)$ 时，前三个 S 盒的输入差分为 0，并且

$$T[0][0] = 1.$$

只有最后一个 S 盒产生分支。对于本实验使用的 S 盒，输入差分 1 可以传播为输出差分 1、2、5，其概率分别为 1/2、1/4、1/4。因此，经过 S 盒层后可能得到

$$0x0001 (1/2), \quad 0x0002 (1/4), \quad 0x0005 (1/4).$$

代码中的 `score` 保存四个半字节的联合概率分布，其形状为

$$(\text{batch}, 16, 16, 16, 16).$$

处理某一个半字节时，程序计算

$$F_{\text{new}}(\beta) = \sum_{\alpha=0}^{15} F_{\text{old}}(\alpha) T[\alpha][\beta].$$

这对应代码中的矩阵乘法：

```

1 score_matrix = moved_score.reshape(-1, 16)
2 updated_score = score_matrix @ probability_table

```

程序依次处理四个 S 盒：

```

1 for position in range(4):
2     score = update_one_sbox_distribution(
3         score, probability_table, position
4     )

```

对于一个确定的当前 16 比特差分，这一过程等价于四个 S 盒概率的乘积；当当前状态本身是多个差分状态的概率混合时，程序还会对所有前驱状态的概率进行求和，因此需要保存四个半字节的联合分布，而不能只分别保存四个边缘概率。

四个 S 盒处理完成后，将得到的 16 比特差分经过 P 置换。代码预先计算所有 16 比特状态的置换结果：

```

1 permutation = np.array(
2     [P_box(value) for value in range(STATE_SIZE)],
3     dtype=np.int32,
4 )

```

由于 P 是比特置换，满足

$$P(X) \oplus P(Y) = P(X \oplus Y),$$

因此可以直接对差分状态进行 P 置换。经过 P 置换后的差分分布作为下一轮的输入。

4.4 四轮动态规划

设初始输入差分为 α ，经过 r 轮后的差分分布为 $D_r^{(\alpha)}$ 。初始时只有输入差分本身的概率为 1：

$$D_0^{(\alpha)}(\Delta) = \begin{cases} 1, & \Delta = \alpha, \\ 0, & \Delta \neq \alpha. \end{cases}$$

代码通过以下语句完成初始化：

```

1 score[rows, nibble0, nibble1, nibble2, nibble3] = 1.0

```

记一轮四个 S 盒和 P 置换的差分转移概率为 $R(\Delta, \Delta')$ ，则每轮动态规划满足

$$D_{r+1}^{(\alpha)}(\Delta') = \sum_{\Delta} D_r^{(\alpha)}(\Delta) R(\Delta, \Delta').$$

因此，四轮后的输出差分概率为

$$D_4^{(\alpha)}(\beta) = \sum_{\Delta_1, \Delta_2, \Delta_3} R(\alpha, \Delta_1) R(\Delta_1, \Delta_2) R(\Delta_2, \Delta_3) R(\Delta_3, \beta).$$

这个求和包含了从输入差分 α 到输出差分 β 的所有差分路线。代码的外层循环对应四轮传播：

```

1  for _ in range(NUMBER_OF_ROUNDS):
2      for position in range(4):
3          score = update_one_sbox_distribution(
4              score, probability_table, position
5          )
6
7      after_sbox = score.reshape(batch_size, STATE_SIZE)
8      after_pbox[:, permutation] = after_sbox
9      score = after_pbox.reshape(
10         batch_size, 16, 16, 16, 16
11     )

```

这里每一轮的输入和输出都是 16 比特差分状态的概率分布，不是具体的明文或密文。四轮循环结束后，score 中保存每个输入差分经过四轮后得到所有输出差分的概率。

4.5 高概率差分的搜索

程序从 0x0001 到 0xFFFF 遍历全部 $2^{16} - 1$ 个非零输入差分，并将输入差分按 32 个一批进行处理，以降低内存开销。分批处理不会减少搜索范围。

对于每个输入差分 α ，四轮动态规划得到所有输出差分 β 的概率分布。程序使用

```

1  output_differences = np.argmax(
2      output_distribution, axis=1
3  )

```

找到该输入差分对应的最大概率输出差分。随后，再在所有输入差分对应的候选结果中选择全局最大值。

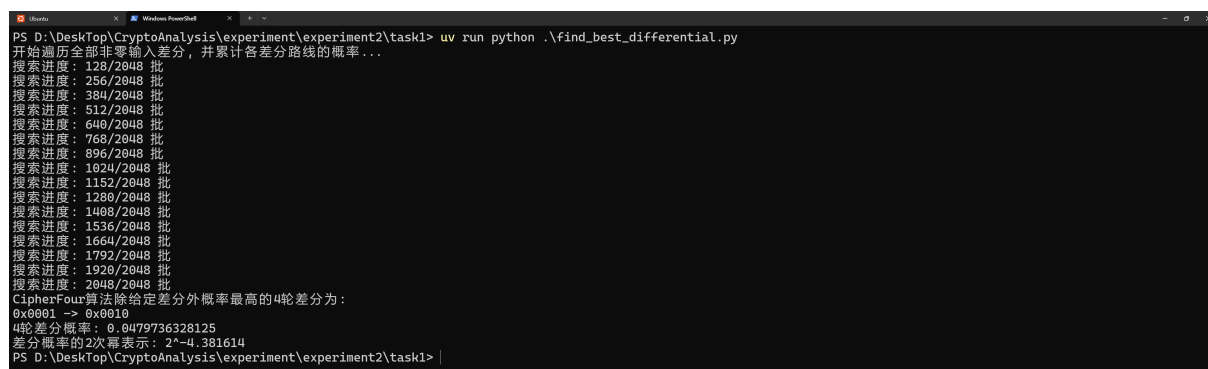
题目要求排除 $0x0001 \rightarrow 0x0001$ ，因此代码只对输入差分 0x0001 的输出分布执行：

```

1  candidate_distribution[
2      EXCLUDED_OUTPUT_DIFFERENCE
3  ] = 0.0

```

然后重新选择该输入差分对应的最佳输出差分。



```

PS D:\Desktop\CryptoAnalysis\experiment\experiment2\task1> uv run python .\find_best_differential.py
开始遍历全部非零输入差分，并累计各差分路线的概率...
搜索进度: 128/2048 批
搜索进度: 256/2048 批
搜索进度: 384/2048 批
搜索进度: 512/2048 批
搜索进度: 640/2048 批
搜索进度: 768/2048 批
搜索进度: 896/2048 批
搜索进度: 1024/2048 批
搜索进度: 1152/2048 批
搜索进度: 1280/2048 批
搜索进度: 1408/2048 批
搜索进度: 1536/2048 批
搜索进度: 1664/2048 批
搜索进度: 1792/2048 批
搜索进度: 1920/2048 批
搜索进度: 2048/2048 批
CipherFour算法除给定差分外概率最高的4轮差分为:
0x0001 -> 0x0010
4轮差分概率: 0.0479736328125
差分概率的2次幂表示: 2^-4.381614
PS D:\Desktop\CryptoAnalysis\experiment\experiment2\task1>

```

Figure 2: 寻找高概率差分

程序最终找到的四轮高概率差分为

$$(0x0, 0x0, 0x0, 0x1) \rightarrow (0x0, 0x0, 0x1, 0x0),$$

即

$$0x0001 \rightarrow 0x0010.$$

其基于 DDT 动态规划得到的四轮差分概率为

$$P(0x0001 \rightarrow 0x0010) = 0.0479736328125 \approx 2^{-4.381614}.$$

这里计算的是连接相同输入、输出差分的所有差分路线的概率总和，而不是某一条单独差分路线的概率。后续任务二再通过枚举具体明文对，统计该输入、输出差分在固定 CipherFour 置换网络中的实际路线和频率。

4.6 方法 2

运行给的示例程序 3.2 就找到了 0001 -> 0001 -> 0001 -> 0001 -> 0001 这条路线

这条路线就是全局下的最优解不假，但是给的示例程序问题就是只找到了这么一条路线

不过还有可能有与他概率相等的另一条路线

一开始思路是，每一轮多保存几个差分，最后笛卡尔积，串起来，但那样好像挺麻烦，当我盯着示例程序的时候，有了一个想法

我们这里把输入差分为 0 的初始概率设成 0，就是因为它必然以概率为 1 得到输出差分为 0，所以你如果把这个也设成 1

那么最后找到的最优路线肯定是全 0

```
lab2 > 第3章教材二维码对应程序 > task1.py > search_best_route
101 def search_best_route():
113     # 提前计算全部16比特状态的P置换结果
114     permutation = np.array([P_box(x) for x in range(2 ** 16)])
115
116     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
117     best_probability = []
118     # 第一个索引是看哪一轮，第二个索引看输出差分
119     # 允许所有非零输入差分，初始概率设为1
120     first_round_input = np.ones(2 ** 16)
121     first_round_input[0] = 1.0 #输入差分为0直接把概率设成0，这样不用考虑这条路径了
122     # first_round_input[1] = 0.0
123     best_probability.append(first_round_input)
124     # 向前计算4轮的最大概率
125     for round_index in range(4):
126         score = best_probability[round_index].reshape(16, 16, 16, 16) #把输出差分分成4个部分，16
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Figure 3: 概率设成 1

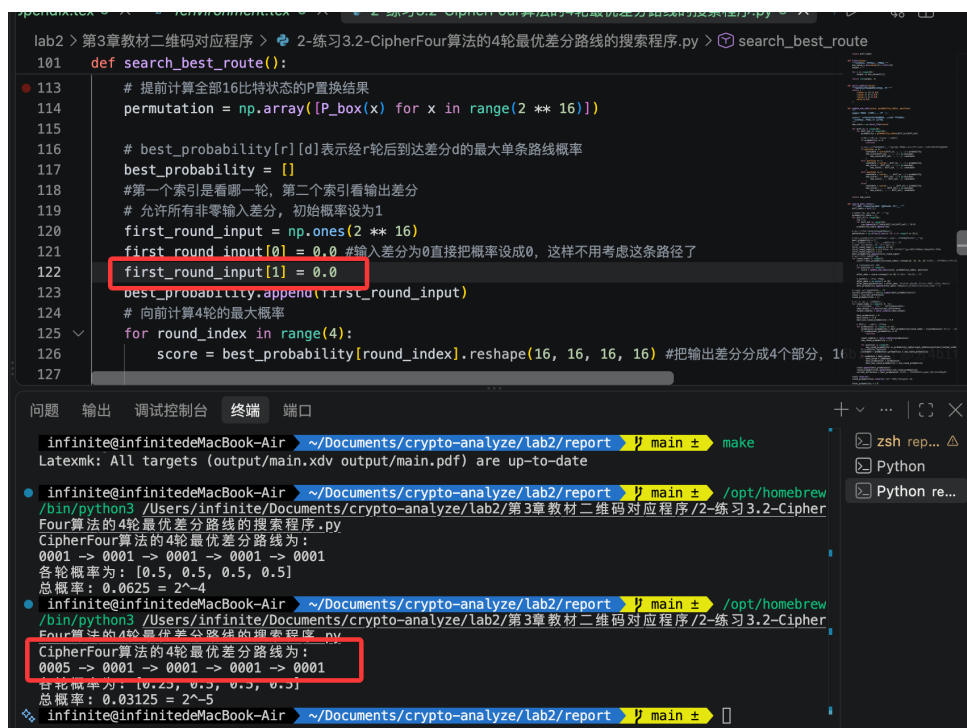
换言之，初始概率为 0 就能屏蔽这条路线，前面就是通过把输入为 0 排除掉，从而得到了 0001 这条路线

那我如果要找另一条路线，再把 0001 屏蔽掉，不就好了吗？

两个路线相同，肯定是每一步都相同，那说句废话，两个路线不同，自然就是有一步不同就行

所以我们每一轮都尝试屏蔽 0001，然后找其中概率最大的，就是题目要求的不同于 0001 这条路线的最好情况了

4.7 第一轮不同



```
lab2 > 第3章教材二维码对应程序 > 2-练习3.2-CipherFour算法的4轮最优差分路线的搜索程序.py > search_best_route
101 def search_best_route():
113     # 提前计算全部16比特状态的P置换结果
114     permutation = np.array([P_box(x) for x in range(2 ** 16)])
115
116     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
117     best_probability = []
118     #第一个索引是看哪一轮，第二个索引看输出差分
119     # 允许所有非零输入差分，初始概率设为1
120     first_round_input = np.ones(2 ** 16)
121     first_round_input[0] = 0.0 #输入差分为0直接把概率设成0，这样不用考虑这条路径了
122     first_round_input[1] = 0.0
123     best_probability.append(first_round_input)
124     # 向前计算4轮的最大概率
125     for round_index in range(4):
126         score = best_probability[round_index].reshape(16, 16, 16, 16) #把输出差分分成4个部分，16
127
问题 输出 调试控制台 终端 端口
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report $ python main.py
Latexmk: All targets (output/main.xdv output/main.pdf) are up-to-date
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report $ python main.py /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/2-练习3.2-Cipher
Four算法的4轮最优差分路线的搜索程序.py
CipherFour算法的4轮最优差分路线为：
0001 -> 0001 -> 0001 -> 0001 -> 0001
各轮概率为：[0.5, 0.5, 0.5, 0.5]
总概率：0.0625 = 2^-4
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report $ python main.py /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/2-练习3.2-Cipher
Four算法的4轮最优差分路线的搜索程序.py
CipherFour算法的4轮最优差分路线为：
0005 -> 0001 -> 0001 -> 0001 -> 0001
各轮概率为：[0.25, 0.5, 0.5, 0.5]
总概率：0.03125 = 2^-5
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report $ python main.py
```

Figure 4: 初始禁用 0001

可以看到找到了 0005 -> 0001 -> 0001 -> 0001 -> 0001 这条路线
这个不好

4.8 找到其他路线

嗯，很多时候有了开始，后面就很简单了，我们每个位置都禁用一下，给 search_best_route 加个 pos 参数，分别禁用每一次

```

lab2 > 第3章教材二维码对应程序 > task1.py > ...
101 def search_best_route(pos):
102     # 将DDT计数转换为S盒差分传播概率
103     probability_table = []
104     for diff_in in range(16):
105         row = []
106         for diff_out in range(16):
107             row.append(diff_table[diff_in][diff_out] / 16.0)
108         probability_table.append(row)
109
110     # 提前计算全部16比特状态的P置换结果
111     permutation = np.array([P_box(x) for x in range(2 ** 16)])
112
113     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
114     best_probability = []
115     # 第一个索引是看哪一轮, 第二个索引看输出差分
116     # 允许所有非零输入差分, 初始概率设为1
117     first_round_input = np.ones(2 ** 16)
118     first_round_input[0] = 0.0 # 输入差分为0直接把概率设成0, 这样不用考虑这条路径了
119     if pos == 0:
120         first_round_input[1] = 0.0
121     best_probability.append(first_round_input)
122
123     # 向前计算4轮的最大概率
124     for round_index in range(4):
125         score = best_probability[round_index].reshape(16, 16, 16, 16) # 把输出差分分成4个部分, 16bit每个部分4bit也就是0-15
126
127         # 依次处理4个并行S盒
128         for position in range(4):
129             score = update_one_sbox(score, probability_table, position)
130
131         after_sbox = score.reshape(2 ** 16) # 再弄回一整个输出差分
132
133         # 将S盒输出差分经过P置换
134         after_pbox = np.zeros(2 ** 16)
135         after_pbox[permutation] = after_sbox # 让after_pbox的第P[i]个位置等于after_sbox[i]
136         best_probability.append(after_pbox) # 得到best_probability[round_index + 1]
137     if round_index + 1 == pos:
138         best_probability[round_index + 1][1] = 0.0 # 禁用输入差分0001
139
140     # 找到4轮后概率最大的输出差分

```

Figure 5: 修改代码

遗憾的是, 没有和前面给的概率相等的路线, 最多只有概率为 $1/32$ 的了

```

infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report 主 main ± /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/task1.py
第0轮不为0001
CipherFour算法的4轮最优差分路线为:
0005 -> 0001 -> 0001 -> 0001 -> 0001
各轮概率为: [0.25, 0.5, 0.5, 0.5]
总概率: 0.03125 = 2^-5

第1轮不为0001
CipherFour算法的4轮最优差分路线为:
1000 -> 0008 -> 0001 -> 0001 -> 0001
各轮概率为: [0.5, 0.25, 0.5, 0.5]
总概率: 0.03125 = 2^-5

第2轮不为0001
CipherFour算法的4轮最优差分路线为:
0001 -> 0001 -> 0010 -> 0002 -> 1001
各轮概率为: [0.5, 0.25, 0.5, 0.375]
总概率: 0.0234375 = 2^-5

第3轮不为0001
CipherFour算法的4轮最优差分路线为:
0001 -> 0001 -> 0001 -> 0010 -> 0002
各轮概率为: [0.5, 0.5, 0.25, 0.5]
总概率: 0.03125 = 2^-5

第4轮不为0001
CipherFour算法的4轮最优差分路线为:
0001 -> 0001 -> 0001 -> 0010 -> 0002
各轮概率为: [0.5, 0.5, 0.25, 0.5]
总概率: 0.03125 = 2^-5

```

Figure 6: 结果

找到了五条路线, 概率都一样, 不过题目要求只看 $\Delta_{in} \rightarrow \Delta_{out}$
那么其实就是四条
0005->0001

1000->0001
0001->1001
0001->0002

5 任务二

根据任务一找到的新 4 轮高概率差分，要寻找所有对应的差分路线，思路为遍历所有 2^{16} 种输入差分对，即 $x \in \{0, 1, \dots, 2^{16} - 1\}$ 、 $x' = x \oplus \delta_{in}$ ，记录下 4 轮加密过程中每一轮的中间异或结果，当满足输出差分 $y \oplus y' = \delta_{out}$ 时，该路线即为所要求的 4 轮差分的差分路线，将该路线记录下来，最后输出所有差分路线并计算概率 (注意这里按照输入是有序对时共 2^{16} 种满足条件的差分对，若按照无序对考察，则差分路线和总的差分对数量均减半，但最终概率不变)，计算概率为：

$$P = \frac{\#\{x : F^4(x) \oplus F^4(x \oplus \delta_{in}) = \delta_{out}\}}{2^{16}}$$

按照上述思路编写代码，运行结果如下：

```

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2> ls

目录: D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/9/23   11:31           3485 find_all_routes.py

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2> uv run python .\find_all_routes.py
输入差分: 0x0001
目标输出差分: 0x0010
正在计算 CipherFour 的一轮 S 盒与 P 置换查找表...
满足输入、输出差分的有序明文对数: 3204
不同差分路线数: 6
目标差分频率: 0.04888916015625

差分路线 | 明文对计数 | 路线频率
-----
0x0001 -> 0x0001 -> 0x0001 -> 0x0001 -> 0x0010 | 2048 | 0.0312500000
0x0001 -> 0x0001 -> 0x0010 -> 0x0002 -> 0x0010 | 562 | 0.0085754395
0x0001 -> 0x0101 -> 0x0005 -> 0x0001 -> 0x0010 | 296 | 0.0045166016
0x0001 -> 0x0001 -> 0x0101 -> 0x0005 -> 0x0010 | 238 | 0.0036315918
0x0001 -> 0x0101 -> 0x0105 -> 0x0005 -> 0x0010 | 44 | 0.0006713867
0x0001 -> 0x0101 -> 0x0050 -> 0x0002 -> 0x0010 | 16 | 0.0002441406
路线总频率为: 0.04888916015625
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2>

```

Figure 7: 所有新 4 轮差分路线

6 任务三

6.1 实验目标与方法

本任务对任务一选取的四轮差分 $\Delta_{in} = 0x0001 \rightarrow \Delta_{out} = 0x0010$ 进行实验验证。随机选取五组不同的轮密钥，在每组密钥下遍历全部 2^{16} 个明文 x ，构造满足输入差分为 Δ_{in} 的明文对

$$(x, x') = (x, x \oplus \Delta_{in}),$$

分别进行四轮 CipherFour 加密，统计密文差分等于 Δ_{out} 的次数，并计算相应频率及五组频率的平均值。

为使实验结果可以复现，程序使用伪随机数生成器产生轮密钥，并将随机种子固定为 20261001。每组密钥记为

$$K^{(j)} = (K_0^{(j)}, K_1^{(j)}, K_2^{(j)}, K_3^{(j)}, K_4^{(j)}), \quad j = 1, \dots, 5,$$

其中每个密钥字均为 16 比特，且五组密钥互不相同。

四轮加密的每一轮均包含轮密钥异或、S 盒替换和 P 置换，第四轮同样保留 P 置换。设初始状态为 $u_0 = x$ ，则

$$u_{r+1} = P(S(u_r \oplus K_r)), \quad r = 0, 1, 2, 3,$$

最终密文为

$$E_K(x) = u_4 \oplus K_4.$$

其中， S 表示对 16 比特状态的四个半字节并行应用 S 盒。

对于第 j 组密钥，满足目标输出差分的有序明文对数为

$$N_j = \#\{x \in \{0, 1, \dots, 2^{16} - 1\} : E_{K^{(j)}}(x) \oplus E_{K^{(j)}}(x \oplus \Delta_{\text{in}}) = \Delta_{\text{out}}\},$$

相应的差分出现频率为

$$f_j = \frac{N_j}{2^{16}},$$

五组密钥下的平均频率为

$$\bar{f} = \frac{1}{5} \sum_{j=1}^5 f_j.$$

本实验采用有序对计数。由于输入差分非零，每个无序明文对 (x, x') 会以 (x, x') 和 (x', x) 两种顺序各计数一次。若改用无序对计数，则匹配对数与总对数均减半，因此最终得到的频率不变。

6.2 程序实现

程序首先预计算全部 16 比特状态经过 S 盒和 P 置换后的结果，建立查找表

$$\text{SP}[v] = P(S(v)).$$

加密时，每轮通过查表完成 S 盒替换和 P 置换：

```
1 state = plainText
2 for round_index in range(NUMBER_OF_ROUNDS):
3     state = SP_table[state ^ keys[round_index]]
4 return state ^ keys[NUMBER_OF_ROUNDS]
```

对于每组密钥，程序先计算全部明文对应的密文，然后遍历明文 x ，通过 $x \oplus \Delta_{\text{in}}$ 定位配对明文，判断对应的密文差分是否为目标输出差分。核心统计过程如下：

```
1 count = 0
2 for plainText in range(STATE_SIZE):
3     paired_plainText = plainText ^ INPUT_DIFFERENCE
4     if (
5         cipherText[plainText] ^ cipherText[paired_plainText]
6         == OUTPUT_DIFFERENCE
7     ):
8         count += 1
9
10 frequency = count / float(STATE_SIZE)
```

这种方法对每组密钥均覆盖全部 2^{16} 个明文，得到该固定密钥下目标差分的实际出现频率。

6.3 实验结果

本次实验随机生成的五组轮密钥如表 1 所示，各组密钥下的匹配对数及频率如表 2 所示。表中密钥均采用十六进制表示。

Table 1: 任务三使用的五组轮密钥

组别	K_0	K_1	K_2	K_3	K_4
1	7998	A426	B83F	C8AA	3F89
2	4CC3	605E	27E6	45C7	1314
3	5F96	ECF1	C21E	6D7E	AE5A
4	620C	890A	8237	C80D	6808
5	A00C	0D2F	7645	7F31	86AC

Table 2: 不同密钥下目标输出差分的出现频率

组别	匹配有序明文对数 N_j	频率 f_j
1	3144	0.0479736328
2	3124	0.0476684570
3	3120	0.0476074219
4	3196	0.0487670898
5	3018	0.0460510254
平均	3120.4	0.0476135254

程序运行结果如图 8 所示。

```
wjy@192 crypto-analyze % /usr/local/bin/python3 /Users/wjy/大三/实验/crypto-analyze/lab2/task3/test_differential_frequency.py
输入差分: 0x1
目标输出差分: 0x10
密钥组数: 5
随机种子: 20261001

序号 | 轮密钥组 | 匹配对数 | 输出差分频率
-----|-----|-----|-----
1 | [0x7998, 0xA426, 0xB83F, 0xC8AA, 0x3F89] | 3144 | 0.0479736328
2 | [0x4CC3, 0x605E, 0x27E6, 0x45C7, 0x1314] | 3124 | 0.0476684570
3 | [0x5F96, 0xECF1, 0xC21E, 0x6D7E, 0xAE5A] | 3120 | 0.0476074219
4 | [0x620C, 0x890A, 0x8237, 0xC80D, 0x6808] | 3196 | 0.0487670898
5 | [0xA00C, 0x0D2F, 0x7645, 0x7F31, 0x86AC] | 3018 | 0.0460510254

平均输出差分频率: 0.0476135254
```

Figure 8: 五组随机密钥下的四轮差分频率测试结果

五组密钥下的平均频率为

$$\bar{f} = \frac{3144 + 3124 + 3120 + 3196 + 3018}{5 \times 65536} = 0.047613525390625,$$

即目标输出差分的平均出现频率约为 4.7614%。

6.4 结果分析

按 S 盒差分分布表计算并累加各条差分路线概率，所选四轮差分的理论概率为

$$P_{\text{DDT}} = 0.0479736328125.$$

本次实验的平均频率与该理论概率的绝对差为

$$|\bar{f} - P_{\text{DDT}}| = 0.000360107421875,$$

相对偏差为

$$\frac{|\bar{f} - P_{\text{DDT}}|}{P_{\text{DDT}}} \times 100\% \approx 0.7506\%.$$

可见，本次五组密钥下的平均频率与 DDT 理论概率较为接近。

各组密钥下的频率并不完全相同，其中第四组最高，约为 4.8767%，第五组最低，约为 4.6051%。虽然同一轮密钥在一对状态的异或中会抵消，但轮密钥会改变进入 S 盒的具体取值，进而影响后续各轮差分传播的实际计数。因此，固定密钥下的差分频率可以随密钥变化。

DDT 路线概率的乘积及其求和对应独立均匀轮密钥条件下的平均概率，而本实验仅选取五组具体密钥，其平均频率不必与理论概率完全相等。同时，每组密钥下已穷举全部明文，因此这里的差异不是由明文抽样不充分造成的。

实验结果表明，在本次选取的五组密钥下，差分 $0x0001 \rightarrow 0x0010$ 均保持较高的出现频率，其平均值与理论计算接近，为任务一所选高概率差分提供了实验验证。

7 任务四

7.1 实验目标与攻击思路

本任务在给定的第 3.1.2.4 节五轮 CipherFour 差分密钥恢复攻击程序上进行修改，保留读取数据、构造明文对、去噪、逆 S 盒部分解密与候选计数的流程，使用任务一所选的四轮差分

$$\Delta_{\text{in}} = 0x0001 \longrightarrow \Delta_{\text{out}} = 0x0010$$

对五轮 CipherFour 进行差分密钥恢复攻击。攻击使用给定的 `plaintext.txt` 和 `ciphertext.txt` 中一一对应的明密文，恢复最后一个轮密钥 K_5 的一个 4 比特半字节。

五轮算法使用六个 16 比特轮密钥 $K_0, \dots, K_5$ 。前四轮包含轮密钥异或、S 盒替换及 P 置换，第五轮不包含 P 置换。设前四轮的输出为 U_4 ，则最终密文为

$$C = S(U_4 \oplus K_4) \oplus K_5.$$

其中， S 表示对四个半字节并行应用 S 盒。

目标四轮输出差分为 $0x0010$ ，因此第五轮只有从左到右的第三个 S 盒活跃，其输入差分为 1。将半字节从左到右编号为 0, 1, 2, 3，攻击目标为 $k_{5,2}$ ，其提取方式为

$$k_{5,2} = (K_5 \gg 4) \& 0xF.$$

7.2 明文配对与去噪条件

首先建立明文到密文的对应表，对每个明文 x 查找 $x' = x \oplus 0x0001$ ，得到对应的密文对 (C, C') 。实验采用有序对计数，对完整的 16 比特码本共得到 2^{16} 个有序对。

根据本实验 S 盒的差分分布表，输入差分为 1 时，可能的输出差分为

$$\{1, 2, 5\}.$$

对于满足四轮输出差分 $0x0010$ 的正确对，第五轮三个非活跃 S 盒的输出差分仍为零，活跃 S 盒的输出差分属于上述集合。由于第五轮无 P 置换，最终密文差分必须满足

$$C \oplus C' \in \{0x0010, 0x0020, 0x0050\}.$$

程序据此筛选密文对。等价地，可以先检查

$$(C \oplus C') \& 0xFF0F = 0,$$

再检查活跃半字节的差分是否属于 $\{1, 2, 5\}$ 。

该条件是正确对必须满足的必要条件，因此正确对都会保留；部分不满足目标四轮差分的错误对也可能通过筛选，需要在后续密钥评分时处理。

7.3 六组密钥下的去噪实验

首先使用题目指定的六个轮密钥

$$(0123, 4567, 89AB, CDEF, FEDC, BA98),$$

随后使用随机种子 20261001 另外生成五组不同的密钥。每组密钥均包含六个 16 比特密钥字，具体取值见图 9。

对每组密钥，穷举全部 2^{16} 个有序明文对，记录第四轮输出和最终密文。记去噪后保留的对数为 M_j ，其中满足第四轮输出差分 $0x0010$ 的正确对数为 T_j ，则正确对占比为

$$\rho_j = \frac{T_j}{M_j}.$$

这里的 T_j 通过已知密钥的加密仿真及第四轮中间状态获得，仅用于评估去噪效果；对给定明密文进行密钥攻击时，评分程序不使用这些中间状态。程序运行结果如图 9 所示。

```

wly@192 crypto-analyze % /usr/local/bin/python3 /Users/wly/大三/实验/crypto-analyze/lab2/task4/task4_denoising_test.py
输入差分: 0x0001, 四轮目标输出差分: 0x0010
五轮算法最后一轮无P置换, 六个16比特轮密钥
随机种子: 20261001
组别 | 轮密钥组 | 去噪保留对数 | 正确对数 | 正确对占比
-----|-----|-----|-----|-----
0 | [0x0123, 0x4567, 0x89AB, 0xCDEF, 0xFEDC, 0xBA98] | 4028 | 3172 | 0.7874875869
1 | [0x7998, 0xA426, 0xB83F, 0xC8AA, 0x3F89, 0x4CC3] | 3836 | 3144 | 0.8196037539
2 | [0x605E, 0x27E6, 0x45C7, 0x1314, 0x5F96, 0xECF1] | 3862 | 3140 | 0.8130502330
3 | [0xC21E, 0x6D7E, 0xAE5A, 0x620C, 0x890A, 0x8237] | 3874 | 3100 | 0.8002065049
4 | [0xC80D, 0x6808, 0xA00C, 0x0D2F, 0x7645, 0x7F31] | 3662 | 3052 | 0.8334243583
5 | [0x86AC, 0xF5B2, 0xE7BE, 0x5EC6, 0xB863, 0xC6C5] | 3824 | 3108 | 0.8127615063
每组初始有序明文对数: 65536
各组正确对占比的平均值: 0.8110889905
合并六组后的正确对占比: 0.8107077883

```

Figure 9: 六组密钥下的去噪测试结果

六组正确对占比的算术平均为 **81.1089%**，若合并六组实验的全部保留对，则正确对占比为 **81.0708%**，两种统计均表明，去噪后保留的明文对中约有 81% 满足目标四轮差分。

7.4 最后一轮部分解密与密钥评分

提取密文的活跃半字节

$$c = (C \gg 4) \& 0xF, \quad c' = (C' \gg 4) \& 0xF.$$

对 $h \in \{0, 1, \dots, 15\}$ 逐一猜测 $k_{5,2}$, 计算

$$v = S^{-1}(c \oplus h), \quad v' = S^{-1}(c' \oplus h).$$

若 $v \oplus v' = 1$, 则该猜测得分加一。由于第五轮输入端的 K_4 在差分中抵消, 这个判别不需要猜测 K_4 。

给定文件包含全部 65536 个明文及对应密文。主差分去噪后保留 4028 个有序对, 16 种密钥猜测的得分如表 3 所示。

Table 3: 给定明密文的主差分密钥评分

猜测值	得分	猜测值	得分
0	1544	8	3172
1	1544	9	3172
2	1268	A	1468
3	1268	B	1468
4	1232	C	1902
5	1232	D	1902
6	956	E	198
7	956	F	198

最高分候选为 $\{0x8, 0x9\}$, 二者得分均为 3172。本实验 S 盒在输入差分 1 的判别上存在对称性: 对于任意 a, b ,

$$S^{-1}(a) \oplus S^{-1}(b) = 1 \iff S^{-1}(a \oplus 1) \oplus S^{-1}(b \oplus 1) = 1.$$

因此 h 与 $h \oplus 1$ 对每一对密文都会给出相同的主差分判别结果。这种并列无法仅通过增加同类明文对来消除; 主差分攻击只能将候选范围从 16 个缩小为 2 个, 不能单独唯一确定全部 4 比特。

7.5 复杂度分析

设给定明密文条目数为 N , 去噪后保留对数为 M 。建立明密文对应表、构造输入差分对及去噪扫描需要 $O(N)$ 时间。枚举 $2^4 = 16$ 个目标半字节猜测, 对每个保留对进行逆 S 盒部分解密, 评分需要 $O(16M)$ 时间。因此攻击总时间复杂度为

$$O(N + 16M).$$

本次 $N = 65536$ 、 $M = 4028$, 共进行

$$16 \times 4028 = 64448$$

次候选判别，每次使用两次逆 S 盒查表，即 128896 次逆 S 盒查表。这些是半字节部分解密操作，不等于完整五轮加密的次数。

本次使用全部 2^{16} 个不同明文的密文，这表示实验实际使用的数据量，并非攻击所需最少数据量的结论。有序对计数得到 2^{16} 对，不同无序明文对只有 2^{15} 对。若按无序对去重，所有计数及评分开销均减半，候选排序不变。程序保存明密文对应表和筛选后的密文对，空间复杂度为 $O(N)$ 。

六组密钥的去噪仿真另需进行 6×2^{16} 次五轮加密。该部分用于实验评估，应与读取给定明密文后的攻击成本区分。

8 实验总结

A 实验代码

本附录列出本次实验中使用的 Enigma 加密及性能测试程序。

A.1 寻找差分路线

```
1  """
2  练习3.2 CipherFour算法的4轮最优差分路线的搜索程序
3
4  搜索思路：
5      (1) 首先计算S盒的差分分布表DDT.
6      (2) 对所有 $2^{16}-1$ 个非零输入差分同时进行搜索.
7      (3) 每一轮依次计算4个S盒的最大传播概率，再进行P置换.
8      (4) 保存每轮到达各个差分状态的最大概率，最后倒推出完整路线.
9
10  注意：本程序搜索的是概率最高的单条差分路线(differential trail)，不是将相同输入，
11      输出差分之间的多条路线概率相加.
12  """
13  import numpy as np
14
15  # CipherFour算法的S盒定义
16  S = [9, 8, 0, 1, 14, 11, 12, 13, 3, 2, 5, 7, 15, 10, 6, 4]
17
18  # CipherFour算法的P置换定义
19  P = [0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15]
20
21
22  def diff_T():
23      """计算S盒的差分分布表DDT."""
24      diff_table = [[0 for _ in range(16)] for _ in range(16)]
25
26      # 遍历所有输入差分
27      for diff_in in range(16):
28          # 遍历S盒的所有输入
29          for x1 in range(16):
30              x2 = x1 ^ diff_in
31              diff_out = S[x1] ^ S[x2]
32              diff_table[diff_in][diff_out] += 1
33
34      return diff_table
35
36
37  def P_box(value):
38      """对16比特差分状态进行P置换."""
39      bin_value = bin(value)[2:].zfill(16)
40      output = ""
```

```
42
43     for i in range(16):
44         output += bin_value[P[i]]
45
46     return int(output, 2)
47
48
49 def split_nibbles(value):
50     """将16比特状态拆成4个4比特差分."""
51     return [
52         (value >> 12) & 0xF,
53         (value >> 8) & 0xF,
54         (value >> 4) & 0xF,
55         value & 0xF,
56     ]
57
58
59 def update_one_sbox(score, probability_table, position):
60     """
61     对指定位置的一个S盒进行差分传播.
62
63     score是一个16*16*16*16的数组，四个下标分别表示
64     一个16比特差分的四个 半字节.
65     """
66     new_score = np.zeros_like(score)
67
68     for diff_in in range(16):
69         for diff_out in range(16):
70             probability = probability_table[diff_in][diff_out]
71
72             # DDT中计数为0，说明该传播不可能
73             if probability == 0:
74                 continue
75
76             # 对当前半字节取最大路线概率，因为P置换前4个部分是相互独立的，可以分
77             # 别最大化
78             if position == 0:
79                 candidate = score[diff_in, :, :, :] * probability
80                 new_score[diff_out, :, :, :] = np.maximum(
81                     new_score[diff_out, :, :, :], candidate
82                 )
83             elif position == 1:
84                 candidate = score[:, diff_in, :, :] * probability
85                 new_score[:, diff_out, :, :] = np.maximum(
86                     new_score[:, diff_out, :, :], candidate
87                 )
88             elif position == 2:
89                 candidate = score[:, :, diff_in, :] * probability
90                 new_score[:, :, diff_out, :] = np.maximum(
```

```

90         new_score[:, :, diff_out, :], candidate
91     )
92     else:
93         candidate = score[:, :, :, diff_in] * probability
94         new_score[:, :, :, diff_out] = np.maximum(
95             new_score[:, :, :, diff_out], candidate
96         )
97
98     return new_score
99
100
101 def search_best_route(pos):
102     """搜索4轮CipherFour的全局最优单条差分路线."""
103     diff_table = diff_T()
104
105     # 将DDT计数转换为S盒差分传播概率
106     probability_table = []
107     for diff_in in range(16):
108         row = []
109         for diff_out in range(16):
110             row.append(diff_table[diff_in][diff_out] / 16.0)
111         probability_table.append(row)
112
113     # 提前计算全部16比特状态的P置换结果
114     permutation = np.array([P_box(x) for x in range(2 ** 16)])
115
116     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
117     best_probability = []
118     # 第一个索引是看哪一轮，第二个索引看输出差分
119     # 允许所有非零输入差分，初始概率设为1
120     first_round_input = np.ones(2 ** 16)
121     first_round_input[0] = 0.0 #输入差分为0直接把概率设成0，这样不用考虑这条路径了
122     if pos == 0:
123         first_round_input[1] = 0.0
124     best_probability.append(first_round_input)
125     # 向前计算4轮的最大概率
126     for round_index in range(4):
127         score = best_probability[round_index].reshape(16, 16, 16, 16) #把输出差分分成4个部分，16bit每个部分4bit也就是0-15,score就是B_r
128
129         # 依次处理4个并行S盒
130         for position in range(4):
131             score = update_one_sbox(score, probability_table, position)
132
133         after_sbox = score.reshape(2 ** 16) #再弄回一整个输出差分
134
135         # 将S盒输出差分经过P置换
136         after_pbox = np.zeros(2 ** 16)

```

```
137     after_pbox[permutation] = after_sbox #让after_pbox的第P[i]个位置等于
after_sbox[i]
138     best_probability.append(after_pbox) #得到best_probability[round_index
+ 1]
139     if round_index + 1 == pos:
140         best_probability[round_index + 1][1] = 0.0 #禁用输入差分0001
141
142     # 找到4轮后概率最大的输出差分
143     current_difference = int(np.argmax(best_probability[4]))
144     route = [current_difference]
145     round_probabilities = []
146
147     # 从第4轮向第1轮倒推最优路线
148     for round_index in range(4, 0, -1):
149         # P是对合置换, 所以再过一次P即可得到S盒输出
150         sbox_output = P_box(current_difference)
151         output_nibbles = split_nibbles(sbox_output)
152
153         best_predecessor = 0
154         best_value = -1.0
155         best_one_round_probability = 0.0
156
157         # 遍历上一轮的全部差分状态
158         for predecessor in range(2 ** 16):
159             predecessor_probability = best_probability[round_index - 1][
predecessor] #找上一轮的概率
160             if predecessor_probability == 0:
161                 continue
162
163             input_nibbles = split_nibbles(predecessor)
164             one_round_probability = 1.0
165
166             for position in range(4):
167                 one_round_probability *= probability_table[input_nibbles[
position]][output_nibbles[position]]
168             #每个S盒变换概率乘起来
169             candidate = predecessor_probability * one_round_probability
170
171             if candidate > best_value:
172                 best_value = candidate
173                 best_predecessor = predecessor
174                 best_one_round_probability = one_round_probability
175
176             route.append(best_predecessor)
177             round_probabilities.append(best_one_round_probability)
178             current_difference = best_predecessor #把输出差分变成刚刚找到的前驱,
然后重新来
179
180     route.reverse()
```

```
181     round_probabilities.reverse() #倒过来就是输入到输出了
182
183     total_probability = 1.0
184     for probability in round_probabilities:
185         total_probability *= probability #每一轮乘起来
186
187     return route, round_probabilities, total_probability
188
189
190 if __name__ == "__main__":
191     for pos in range(5):
192         print(f"第{pos}轮不为0001")
193         route, round_probabilities, total_probability = search_best_route(pos
194 )
195
196         print("CipherFour算法的4轮最优差分路线为:")
197         print(" -> ".join(hex(x)[2:].zfill(4).upper() for x in route))
198         print("各轮概率为:", round_probabilities)
199         print("总概率:", total_probability, "= 2^" + str(int(np.log2(
200 total_probability))))
201         print()
```

Listing 1: .py

B 实验报告完整地址

实验完整代码、实验报告 L^AT_EX 源代码见仓库: <https://github.com/infinitepwn/crypto-analyze.git>